Minigestor de base de datos multimodal

Informe incremental: Parte 1 (relacional) y Parte 2 (espacial)

Alumno	José Huamaní
Código	202310632
Carrera	Ciencias de la Computación, UTEC
Curso	Base de Datos 2 (2026-2)
Entrega	Entrega parcial, semana 8
Repositorio	`https://github.com/JoseEd0/project-BD2`

Este informe corresponde a la entrega parcial del proyecto y cubre las Partes 1 y 2. El gestor está escrito en Python, sin ORM ni motor de base de datos externo: las estructuras de almacenamiento, los índices y los algoritmos que pide el enunciado están implementados en el proyecto. La Parte 1 incluye el almacenamiento en páginas, los índices B+ y hash, los algoritmos externos, el procesamiento de SQL, las transacciones y la interfaz. La Parte 2 agrega el R-Tree, las consultas espaciales (por rango, k-NN y polígonos) y un panel de mapa. La sección 4 presenta los resultados experimentales; el experimento espacial usa lugares reales de OpenStreetMap y se compara con PostGIS.

1 Diseño arquitectónico

El sistema está organizado en capas (figura 1). Cada capa usa solamente a las que tiene debajo: el motor de consultas no conoce el API y el API no conoce la interfaz. Las estructuras que evalúa el enunciado (páginas, árbol B+, hash extendible, algoritmos externos, parser SQL y R-Tree) están implementadas en el proyecto. Las librerías externas se usan solo para el servidor web (FastAPI), la interfaz (React y Leaflet), las gráficas y las pruebas.

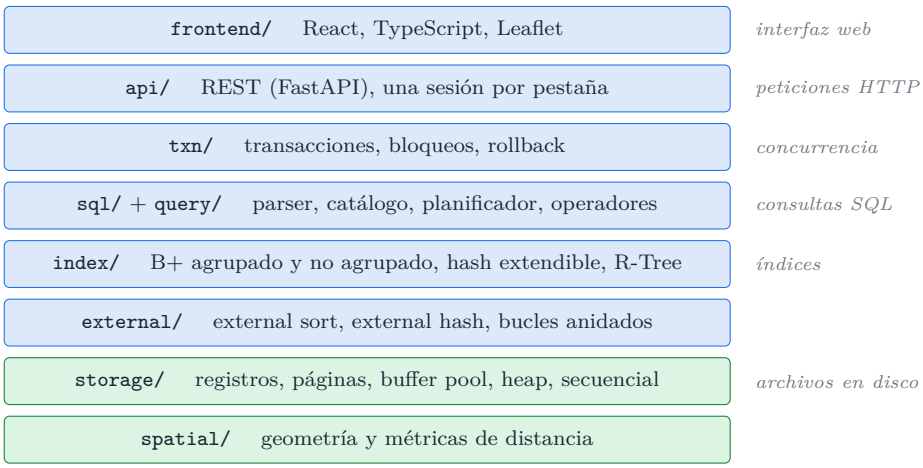


Figura 1: Capas del sistema. Cada bloque es una carpeta del repositorio.

Hay cuatro decisiones de diseño que se aplican en todo el código:

- *Acceso por páginas.* Todas las estructuras leen y escriben páginas de 4 096 bytes a través del mismo paginador. Todos los archivos abiertos comparten un solo buffer pool LRU de 2 048 páginas (8 MiB). Ningún archivo se carga completo en memoria.
- *Registros de tamaño fijo.* Así la posición de un registro dentro de su página se calcula directamente, y la capacidad de cada nodo sale de dividir el tamaño de la página entre el de una entrada.
- *Código compartido.* La serialización de registros, el manejo de páginas y las métricas de distancia están cada una en un solo módulo, que usan tanto los índices como la búsqueda secuencial. Por eso una consulta devuelve lo mismo con índice o sin él.
- *Parámetros configurables.* El tamaño de página, los umbrales y el número de particiones son campos de la clase `EngineConfig`. Las rutas y los puertos se leen de variables de entorno, y los parámetros de los experimentos se pasan por línea de comandos.

Ejecución de una consulta. Una sentencia pasa por las etapas de la figura 2: el lexer y el parser generan el AST, se validan nombres y tipos, el planificador elige el camino de acceso y los operadores producen las filas. Junto con el resultado se devuelve el plan de ejecución, y con `EXPLAIN ANALYZE` también las filas y el tiempo de cada operador.

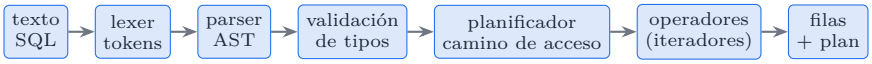


Figura 2: Etapas por las que pasa una sentencia SQL.

API e interfaz. El API REST ejecuta sentencias SQL, lista las tablas, muestra la estructura física de una tabla y de sus índices, y permite crear tablas a partir de un archivo CSV. Cada pestaña del navegador tiene su propia sesión, lo que permite probar los bloqueos con dos pestañas. La interfaz (figura 3) tiene los cuatro paneles que pide el enunciado (archivos, consultas, resultados y plan de ejecución) y, para la Parte 2, un mapa que muestra el resultado, el centro y el radio de la consulta y los MBR del R-Tree.

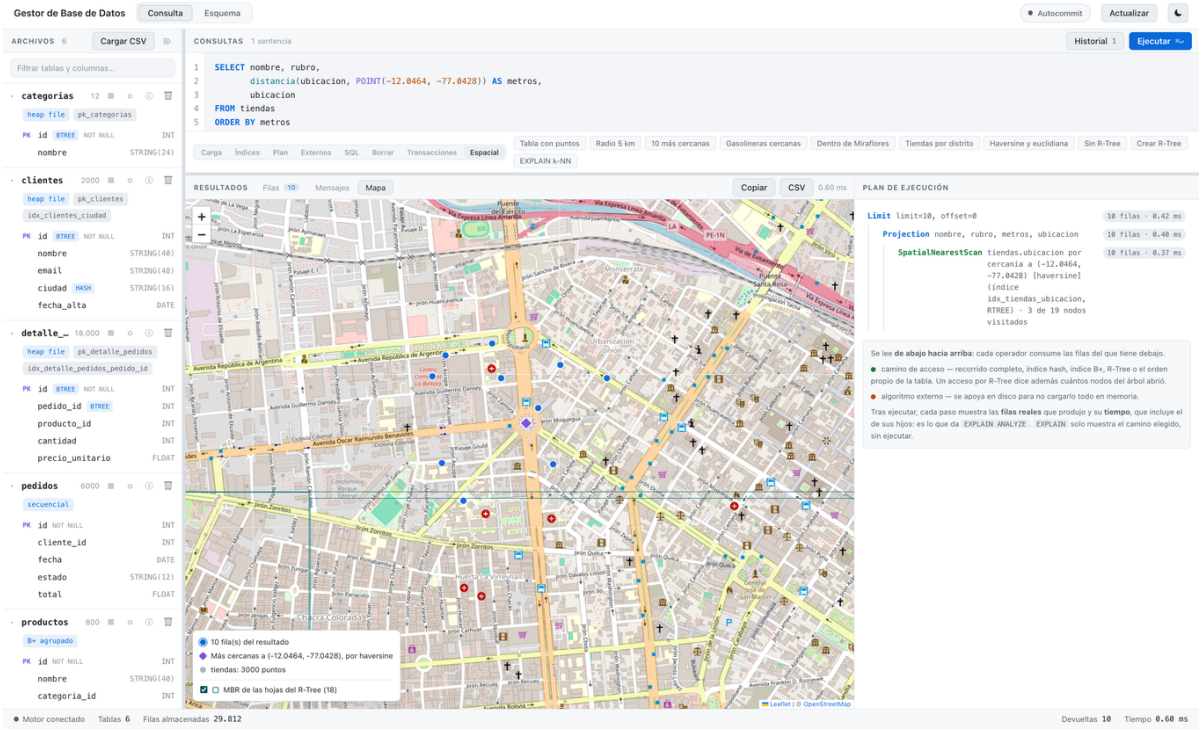


Figura 3: Interfaz después de consultar las 10 tiendas más cercanas a un punto. A la izquierda están las tablas con su organización y sus índices; arriba, el editor SQL y las consultas de ejemplo; al centro, el resultado en el mapa con los MBR de las hojas del R-Tree; a la derecha, el plan de ejecución, que indica que se visitaron 3 de los 19 nodos del índice.

2 Dominio de datos

Se usan dos conjuntos de datos.

Dataset de comercio electrónico (generado). Modela clientes, pedidos, productos y categorías, además de tiendas con ubicación en Lima. Son seis tablas y cerca de 30 000 filas (tabla 1). Las tablas usan distintas organizaciones e índices, para poder mostrar todas las estructuras con los mismos datos.

Los datos se generan con `demos/poblar_ecommerce.py` a partir de una semilla fija, por lo que siempre salen iguales; el tamaño se cambia con la opción `--escala`. Los seis CSV están en el repositorio y se pueden cargar desde la interfaz. Las tiendas se ubican dentro de doce distritos de Lima, lo que permite verificar consultas como «tiendas dentro de Miraflores».

Tabla 1: Tablas del dataset de demostración.

Tabla	Filas	Columnas	Organización	Índice secundario
`categorias`	12	id, nombre	heap file	—
`clientes`	2 000	id, nombre, email, ciudad, fecha_alta	heap file	hash en `ciudad`
`productos`	800	id, nombre, categoria_id, precio, stock	B+ agrupado	—
`pedidos`	6 000	id, cliente_id, fecha, estado, total	archivo secuencial	—
`detalle_pedidos`	18 000	id, pedido_id, producto_id, cantidad, precio_unitario	heap file	B+ en `pedido_id`
`tiendas`	3 000	id, nombre, rubro, distrito, ubicacion	heap file	R-Tree en `ubicacion`

Lugares de OpenStreetMap (reales). Para la parte espacial se necesitaban puntos con una distribución real y no uniforme. El script `demos/descargar_lugares.py` descarga de Geofabrik los puntos de interés del Perú (colegios, restaurantes, farmacias, bancos, gasolineras, etc.) y genera un CSV con las columnas `id`, `nombre`, `rubro` y `ubicacion`, donde la ubicación tiene el formato `POINT(latitud, longitud)`. En total son 128 011 lugares con nombre, de los cuales 26 606 están en Lima y Callao. Este dataset se usa en el experimento espacial y en la demostración del mapa.

Tipos de datos. El gestor soporta enteros, reales, booleanos, fechas, texto de longitud fija, bytes y puntos. Al cargar un CSV, el tipo de cada columna se infiere de sus valores. Las comparaciones solo se permiten entre tipos compatibles: por ejemplo, `WHERE id = 'abc'` da un error que indica la columna, en lugar de devolver cero filas.

3 Explicación de los algoritmos

3.1 Almacenamiento: páginas, heap file y archivo secuencial

Cada registro se guarda como un mapa de nulos seguido de sus campos, todos de tamaño fijo. Una página de 4 096 bytes tiene una cabecera de 8 bytes y luego las ranuras; cada ranura tiene un byte de estado (libre, ocupada o eliminada) y el registro. En una página caben $\lfloor (4096 - 8)/(1 + \text{tamaño del registro}) \rfloor$ registros.

Heap file. Los registros se insertan en cualquier página que tenga espacio. Para encontrarla rápido se mantiene una lista enlazada de las páginas con ranuras libres: la inserción toma la primera página de la lista, y cuando se elimina un registro su página vuelve a la lista. De esta forma se reutiliza el espacio liberado. Buscar por un valor obliga a recorrer todo el archivo, por eso el heap se complementa con índices, que guardan la dirección `(página, ranura)` de cada fila.

Archivo secuencial. Los registros están ordenados por la clave en un espacio principal, y hay un espacio auxiliar para el desbordamiento.

- *Búsqueda:* búsqueda binaria sobre las páginas, $O(\log P)$, luego dentro de la página y, si es necesario, en su cadena de desbordamiento.
- *Inserción:* si la página que corresponde tiene espacio, se desplazan sus registros para mantener el orden. Si está llena y la clave es mayor que todas, se agrega una página al final. En otro caso el registro va a la cadena de desbordamiento de esa página.
- *Eliminación (lazy):* el registro solo se marca como eliminado.
- *Reorganización:* cuando los registros eliminados más los que están en desbordamiento superan el 30 % del total, el archivo se reescribe en orden y sin los eliminados. Cada página se llena al 80 % para dejar espacio a las siguientes inserciones.

3.2 Índices: B+ y hash extendible

Árbol B+. Cada nodo es una página. Los datos están en las hojas, que están enlazadas entre sí para recorrer rangos. Cuando una hoja se llena se divide en dos y la primera clave de la mitad derecha se copia al padre; cuando se llena un nodo interno, la clave del medio sube al padre. Al eliminar, si un nodo queda con menos de la mitad de sus entradas, primero intenta tomar una de un hermano (redistribución) y, si no puede, se fusiona con él. Se implementaron las dos variantes de la tabla 2.

Tabla 2: Variantes del árbol B+. Las entradas por nodo corresponden a páginas de 4 KB y una clave entera (tabla `productos`, con registros de 73 bytes).

		Entradas por nodo	
Variante	**Qué guardan las hojas**	**Hoja**	**Interno**
Agrupado	La fila completa. El árbol es la tabla, así que solo puede haber uno por tabla.	50	340
No agrupado	La dirección de la fila en el heap. La clave es el par `(valor, dirección)` para admitir valores repetidos.	292	226

Hash extendible. Hay un directorio de 2^{gd} punteros a buckets, donde gd es la profundidad global, y cada bucket tiene una profundidad local ld. Una búsqueda lee el directorio y un bucket, sin importar el tamaño de la tabla. Cuando un bucket se llena se divide según el bit ld del hash; si además $ld = gd$, antes se duplica el directorio. Al eliminar, un bucket con pocas entradas se fusiona con su gemelo y, si las dos mitades del directorio quedan iguales, el directorio se reduce a la mitad. Cuando muchas filas tienen la misma clave dividir no sirve, así que en ese caso se encadena una página de desbordamiento. Como función de hash se usa Blake2b, porque el `hash()` de Python cambia en cada ejecución.

3.3 Algoritmos externos

Se usan cuando los datos no caben en memoria. Trabajan con un buffer de 16 páginas y archivos temporales.

External sort (`ORDER BY`). Primero se generan *runs* ordenados del tamaño del buffer y luego se mezclan de 8 en 8 (k-way merge) con una cola de prioridad, manteniendo en memoria una página por run. Se necesitan $\lceil \log_8(N/B) \rceil$ pasadas de mezcla.
External hash (`GROUP BY` y `DISTINCT`). Las filas se reparten en 16 particiones según el hash de la clave, de manera que las filas iguales quedan en la misma partición. Por cada grupo se guarda solo el acumulador de la agregación (un contador, una suma, un mínimo) y no sus filas. Si una partición no cabe en memoria, se vuelve a particionar con otra función de hash.
Grace hash join (`JOIN` con igualdad). Las dos tablas se particionan con la misma función. Luego, por cada par de particiones, una se carga en una tabla hash y la otra se recorre. El costo es $O(N + M)$ en lugar de $O(N \cdot M)$. Soporta claves compuestas y los cuatro tipos de join (`INNER`, `LEFT`, `RIGHT` y `FULL`).
Bucles anidados por bloques (`JOIN` sin igualdad). Si la condición no es una igualdad no se puede particionar, así que se compara cada bloque de la tabla externa contra toda la tabla interna.

3.4 Procesamiento de consultas y transacciones

El parser es recursivo descendente y reporta los errores de sintaxis con línea y columna. El planificador elige el camino de acceso según las condiciones del `WHERE` (tabla 3) y la ejecución sigue el modelo de iteradores (Volcano), en el que cada operador le pide filas al operador que tiene debajo.

Tabla 3: Operador que elige el planificador según la consulta.

Consulta	**Si hay un índice que sirva**	**Si no lo hay**
`col = valor`	`IndexLookup` (hash, B+ o R-Tree)	`SequentialScan`
`col BETWEEN a AND b`	`IndexRange`, `PrimaryKeyRange`	`SequentialScan`
`ORDER BY clave`	`OrderedScan` (la tabla ya está ordenada)	`ExternalSort`
`distancia(col, p) < r`	`SpatialRangeScan`	recorrido y filtro
`intersecta(col, polígono)`	`SpatialPolygonScan`	recorrido y filtro
`ORDER BY distancia(col, p)`	`SpatialNearestScan`	recorrido y `ExternalSort`

Antes de ejecutar se valida la consulta: una columna que no existe, tipos incompatibles o una función mal usada dan error sin leer ninguna fila. Además, `INSERT` y `UPDATE` validan todas las filas antes de escribir, por lo que una sentencia que falla no deja cambios a medias.

Transacciones. Las sentencias entre `BEGIN TRANSACTION` y `END TRANSACTION` forman una transacción. Se usan bloqueos por tabla, compartidos para lectura y exclusivos para escritura, que se liberan al terminar la transacción (bloqueo en dos fases). Para detectar interbloqueos se mantiene un grafo de espera: si esperar un bloqueo formaría un ciclo, esa transacción se aborta. El rollback recorre el registro de cambios en orden

inverso y deshace cada uno. El script `demos/concurrencia.py` muestra esto con hilos: cuatro hilos descuentan 25 veces de un saldo de 1 000. Sin transacciones el saldo termina en 975 en vez de 900, y con transacciones siempre termina en 900. El mismo script provoca un interbloqueo entre dos transacciones y el gestor aborta una de ellas.

3.5 Base de datos espacial: R-Tree

Métricas. Un punto se escribe `POINT(latitud, longitud)`. Se implementaron las dos distancias que pide el enunciado: la euclidiana, que se calcula directamente sobre las coordenadas y queda en grados, y la geodésica (Haversine), que mide la distancia sobre la esfera terrestre en metros:

$$d = 2R \arcsin \sqrt{\sin^2 \tfrac{\Delta\varphi}{2} + \cos\varphi_1 \cos\varphi_2 \sin^2 \tfrac{\Delta\lambda}{2}}, \qquad R = 6\,371\,008.8 \text{ m}$$

Para cada métrica también se implementó la distancia mínima de un punto a un rectángulo, que el R-Tree usa para descartar ramas.

Estructura. El R-Tree se guarda en disco, un nodo por página. Una hoja almacena hasta 186 puntos con la dirección de su fila, y un nodo interno hasta 113 hijos, cada uno con su MBR (el rectángulo mínimo que contiene todo su subárbol). A diferencia del B+, los MBR de nodos hermanos se pueden solapar.

- *Inserción:* se desciende por el hijo cuyo MBR necesita crecer menos. Si la hoja se llena, se divide y se actualizan los MBR hasta la raíz.
- *División cuadrática:* se eligen como semillas las dos entradas que juntas desperdiciarían más área, y las demás se asignan una por una al grupo cuyo MBR crece menos.
- *Eliminación:* si un nodo queda por debajo del 40 % de ocupación, se elimina y sus entradas se vuelven a insertar.
- *Carga masiva (STR):* al crear el índice sobre una tabla que ya tiene datos, los puntos se ordenan por latitud y se dividen en franjas; cada franja se ordena por longitud y se divide en hojas. Para ordenar se reutiliza el external sort de la Parte 1.

Consultas.

Por rango (radio). Se visitan solo los nodos cuyo MBR está a una distancia del centro menor o igual que el radio. En las hojas se calcula la distancia exacta a cada punto.
k-NN. Se usa una búsqueda *best-first* con una cola de prioridad que contiene nodos (con la distancia mínima a su MBR) y puntos (con su distancia real). En cada paso se extrae el elemento más cercano: si es un nodo se agregan sus hijos a la cola, y si es un punto se devuelve como el siguiente vecino. La búsqueda termina al obtener k puntos, sin ordenar el resto.
Intersección con polígonos. Primero se buscan en el árbol los puntos que están dentro del rectángulo que contiene al polígono, y luego se verifica cada uno con la prueba de punto en polígono (regla par-impar), que también funciona con polígonos cóncavos.

```
CREATE INDEX idx_lugares ON lugares USING RTREE (ubicacion);
SELECT * FROM lugares WHERE distancia(ubicacion, POINT(-12.0464, -77.0428)) < 5000;
SELECT * FROM lugares ORDER BY distancia(ubicacion, POINT(-12.0464, -77.0428)) LIMIT 10;
SELECT * FROM lugares WHERE intersecta(ubicacion,
                            POLYGON((-12.11, -77.05), (-12.11, -77.01), (-12.14, -77.01)));
```

Si la columna no tiene un R-Tree, las mismas consultas se resuelven recorriendo la tabla y devuelven el mismo resultado. Cuando se usa el índice, el plan de ejecución indica cuántos nodos del árbol se visitaron.

4 Parte experimental

4.1 Metodología

Cada experimento es un script de `benchmarks/` que recibe por línea de comandos los tamaños, el número de consultas y la semilla, y guarda sus mediciones en `benchmarks/results/` en formato JSON. Las gráficas y las tablas de esta sección se generan a partir de esos archivos. Se usaron los tamaños que pide el enunciado: 1 000, 10 000 y 100 000 registros. Todas las técnicas reciben los mismos datos y las mismas consultas. Las mediciones se hicieron en macOS con Python 3.12 y páginas de 4 KB; para el experimento espacial se usó PostgreSQL 17 con PostGIS 3.6. Los experimentos se ejecutan con:

```
python -m benchmarks.storage_benchmark --sizes 1000 10000 100000 --queries 100
python -m benchmarks.index_benchmark   --sizes 1000 10000 100000 --queries 100
python -m benchmarks.spatial_benchmark --sizes 1000 10000 100000 --queries 100 \
       --points-file data/datasets/lugares_peru.csv
```

Cómo leer las gráficas y las tablas

- Cada gráfica tiene un panel por operación. El eje horizontal es el número de filas de la tabla y el eje vertical es lo que se mide, con su unidad: ms son milisegundos y KiB son kibibytes (1 024 bytes). Cada línea es una técnica y, en todos los casos, un valor más bajo es mejor.
- Los dos ejes están en escala logarítmica, es decir, cada marca vale diez veces la anterior. Una línea horizontal indica que el costo no depende del tamaño de la tabla.
- El número al final de cada línea es el valor medido con 100 000 filas. Las tablas tienen los valores de los tres tamaños.

4.2 Experimento 1: Heap File vs Archivo Secuencial

Sobre la misma tabla se midió el tiempo de insertar todas las filas, 100 búsquedas y 100 borrados por clave primaria, el espacio en disco y el tiempo de reorganización. Cada búsqueda y cada borrado es una operación aparte, como la de un `WHERE id = k`. La reorganización no se forzó: se siguió borrando hasta que el desperdicio pasó del 30 % y se midió el borrado que la disparó. Los resultados están en la figura 4 y en la tabla 4.

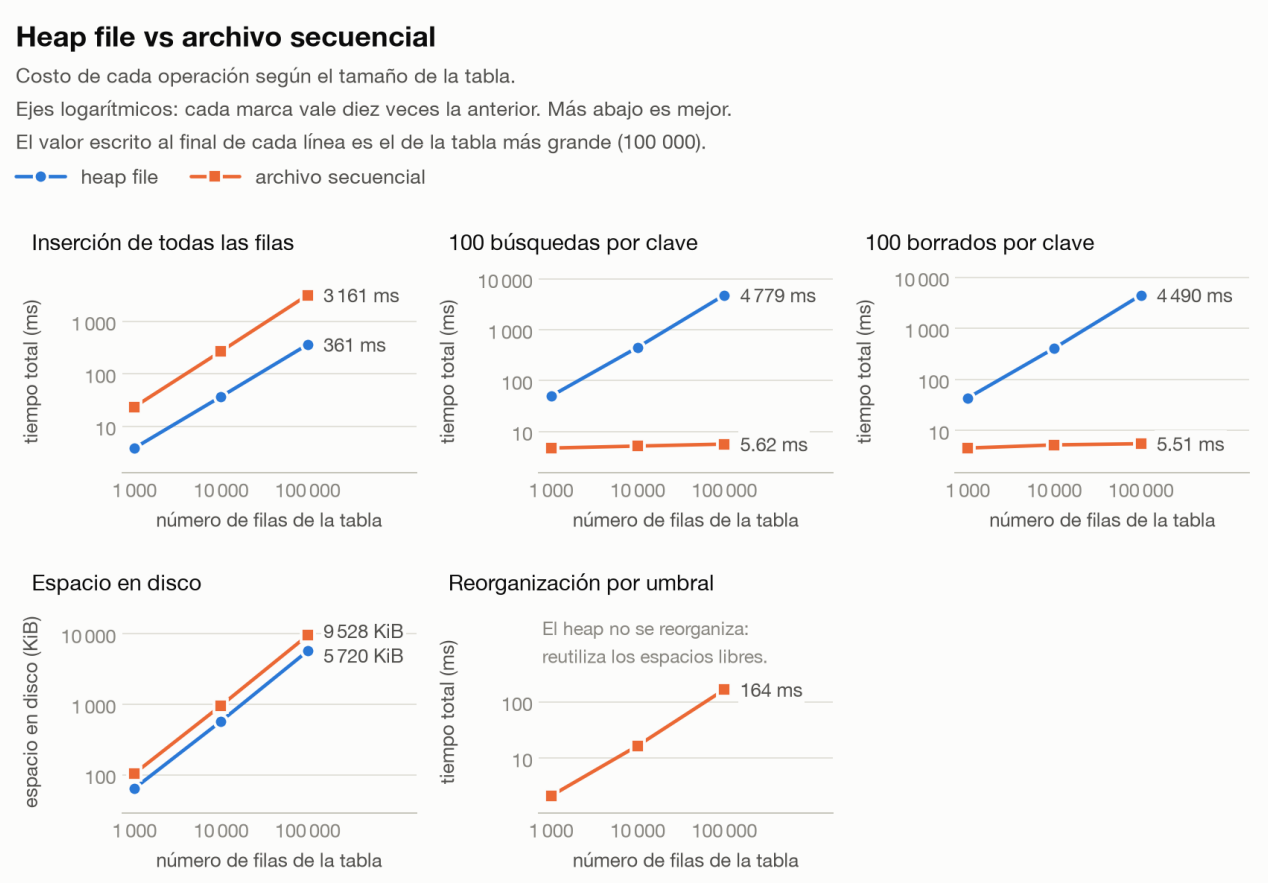


Figura 4: Heap file vs archivo secuencial: inserción, búsqueda y borrado por clave, espacio en disco y reorganización.

- *Búsqueda y borrado.* Sin índice, el heap recorre de media la mitad del archivo por cada clave, así que su tiempo crece en proporción al tamaño, mientras que el del secuencial casi no cambia. Ya con 1 000 filas el secuencial es 11 veces más rápido, y con 100 000 lo es 850 veces al buscar y 815 veces al borrar, porque borrar es encontrar la fila y marcarla.
- *Inserción.* Insertar en el secuencial cuesta 8.8 veces más, porque hay que ubicar la página, desplazar registros y reorganizar el archivo cada cierto tiempo.
- *Reorganización.* Reescribir el archivo con 100 000 filas tomó 164 ms. Es un costo lineal que solo se paga cuando las lápidas y el desbordamiento pasan del 30 % de lo guardado.

- *Espacio.* El secuencial ocupa 1.7 veces más. Esto se debe al factor de llenado del 80 % que se deja después de cada reorganización.

Tabla 4: Resultados del experimento 1. Tiempos en milisegundos (ms) y espacio en KiB. Un valor menor es mejor; el mejor de cada tamaño está en negrita.

		Número de filas de la tabla		
Operación	**Técnica**	**1 000**	**10 000**	**100 000**
Inserción de todas las filas (ms)	heap file	**3.84**	**36.8**	**361**
	archivo secuencial	23.4	268	3 161
100 búsquedas por clave (ms)	heap file	49.5	449	4 779
	archivo secuencial	**4.70**	**5.15**	**5.62**
100 borrados por clave (ms)	heap file	43.3	410	4 490
	archivo secuencial	**4.53**	**5.22**	**5.51**
Espacio en disco (KiB)	heap file	**64.0**	**576**	**5 720**
	archivo secuencial	104	960	9 528
Reorganización al pasar del 30 % (ms)	archivo secuencial	2.12	16.5	164

Tabla 5: Resumen del experimento 1: ventajas, desventajas y cuándo usar cada técnica.

Técnica	**Ventajas**	**Desventajas**	**Cuándo usarla**
Heap file	Inserción en $O(1)$. Ocupa menos espacio. Reutiliza los espacios libres sin reorganizar.	Buscar o borrar por clave recorre el archivo. No mantiene ningún orden.	Tablas con muchas inserciones, que se consultan por medio de un índice.
Archivo secuencial	Búsqueda y borrado por clave en $O(\log P)$: 850 veces más rápido con 100 000 filas. Rangos y recorrido ordenado sin tener que ordenar.	Inserción 8.8 veces más lenta. Ocupa 1.7 veces más. Se reorganiza cuando el desperdicio pasa del 30 %.	Tablas que se consultan por clave o por rango y tienen pocas escrituras.

4.3 Experimento 2: B+ agrupado vs B+ no agrupado vs Hash extendible

Se midió el tiempo de construir el índice, 100 búsquedas por igualdad, 100 búsquedas por rango, el recorrido de toda la tabla en orden, y eliminar y volver a insertar el 10 % de las filas, además del espacio en disco de cada índice (tabla 6 y figura 5). En el B+ no agrupado y en el hash, el tiempo de búsqueda incluye leer la fila del heap.

Tabla 6: Resultados del experimento 2. Tiempos en milisegundos (ms) y espacio en KiB. Un valor menor es mejor; el mejor de cada tamaño está en negrita. *No soportado* indica que el índice no puede resolver esa operación.

		Número de filas de la tabla		
Operación	**Técnica**	**1 000**	**10 000**	**100 000**
Construcción del índice (ms)	B+ agrupado	19.8	298	4 446
	B+ no agrupado	173	2 056	25 587
	hash extendible	**7.45**	**98.4**	**933**
100 búsquedas por igualdad (ms)	B+ agrupado	**1.20**	**2.99**	**3.95**
	B+ no agrupado	12.5	13.0	17.1
	hash extendible	12.5	9.17	12.2
100 búsquedas por rango (ms)	B+ agrupado	**2.48**	**4.43**	**5.49**
	B+ no agrupado	20.3	21.3	35.1
	hash extendible		*no soportado*	
Recorrido ordenado de la tabla (ms)	B+ agrupado	**0.23**	**2.46**	**31.2**
	B+ no agrupado	1.79	18.0	199
	hash extendible		*no soportado*	
Eliminar y reinsertar el 10 % (ms)	B+ agrupado	**4.79**	**86.0**	**1 124**
	B+ no agrupado	43.2	457	5 018
	hash extendible	11.9	92.8	1 159
Espacio en disco (KiB)	B+ agrupado	108	956	9 356
	B+ no agrupado	**24.0**	**220**	**2 040**
	hash extendible	28.0	264	2 056

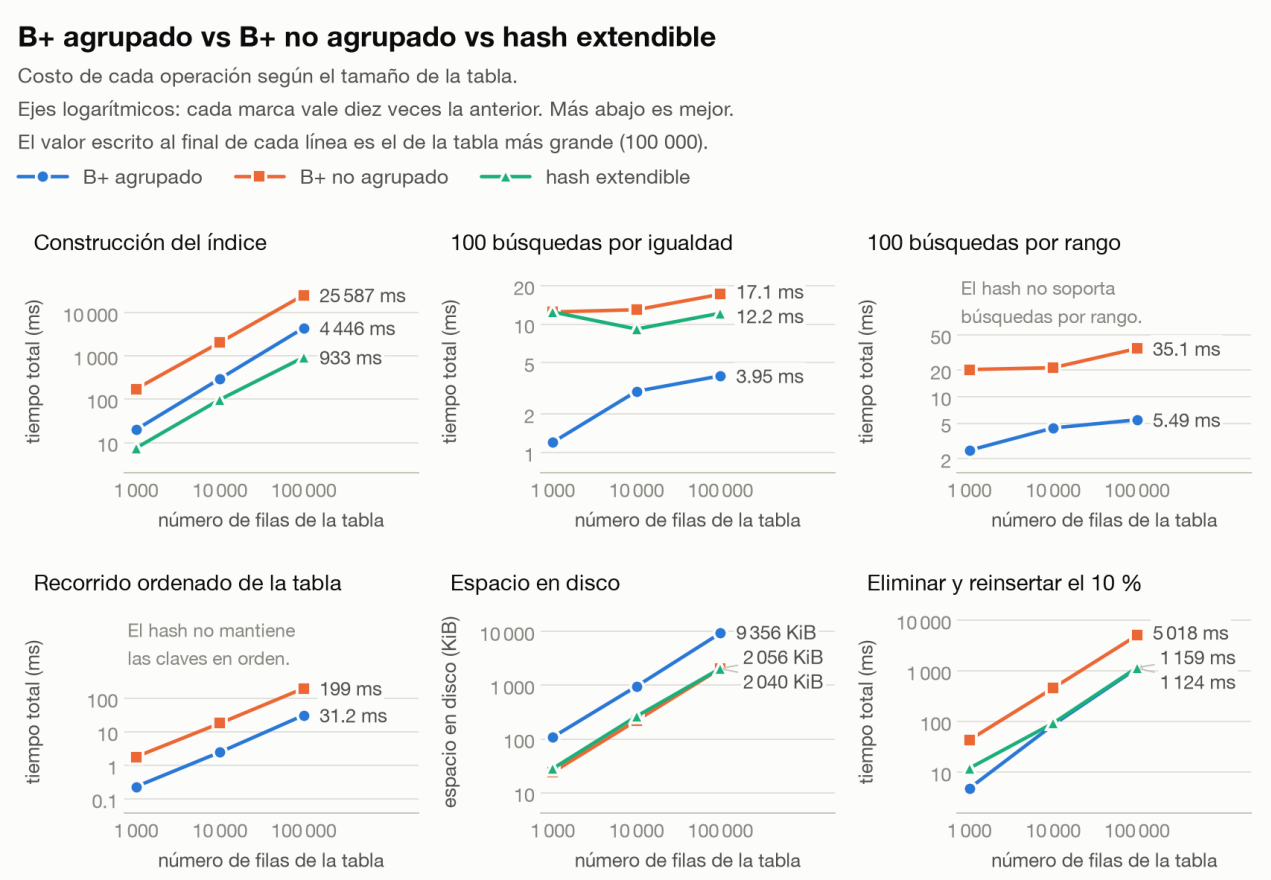


Figura 5: B+ agrupado vs B+ no agrupado vs hash extendible. El hash no aparece en la búsqueda por rango ni en el recorrido ordenado porque no soporta esas operaciones.

- *Búsquedas.* El B+ agrupado es el más rápido: 100 búsquedas por igualdad toman 3.95 ms con 100 000 filas, porque la fila está en la misma hoja. El hash toma 12.2 ms y el B+ no agrupado 17.1 ms; en los dos hay que ir además al heap por la fila.

- *Acceso al heap.* Ese acceso adicional pesa más cuando la consulta devuelve muchas filas. Recorrer las 100 000 filas en orden toma 31.2 ms en el agrupado y 199 ms en el no agrupado, es decir, 6.4 veces más.
- *Construcción y actualizaciones.* El hash es el más rápido de construir (27 veces más rápido que el B+ no agrupado). Al eliminar y reinsertar el 10 % de las filas, en cambio, el B+ agrupado es algo más rápido en los tres tamaños (1 124 ms contra 1 159 ms con 100 000 filas), porque el hash, al borrar, funde cubetas y reduce el directorio. Además, el hash no soporta búsquedas por rango ni recorridos ordenados.
- *Espacio.* El B+ agrupado ocupa 4.6 veces más que los otros dos porque guarda las filas completas.

Tabla 7: Resumen del experimento 2: ventajas, desventajas y cuándo usar cada índice.

Técnica	Ventajas	Desventajas	Cuándo usarla
B+ agrupado	El más rápido en igualdad, rango y recorrido ordenado, porque la fila está en la hoja.	Ocupa 4.6 veces más que los otros. Solo puede haber uno por tabla.	Clave primaria de la tabla.
B+ no agrupado	Soporta rangos y orden. Puede haber varios por tabla y ocupa poco espacio.	Un acceso al heap por cada fila que devuelve. Es el más lento de construir.	Columnas secundarias que se consultan por rango.
Hash extendible	El más rápido de construir (27 veces más que el B+ no agrupado). Igualdad en dos accesos.	No soporta rangos ni recorrido ordenado. También necesita el acceso al heap.	Columnas que solo se consultan por igualdad.

4.4 Experimento 3: Búsqueda Secuencial vs R-Tree vs GiST de PostgreSQL

Las tres técnicas responden las mismas consultas sobre los mismos puntos. Los datos son lugares reales de OpenStreetMap en el Perú, de los que se tomaron muestras de 1 000, 10 000 y 100 000 puntos. Se midieron consultas por rango con radios de 1, 5 y 10 km y consultas k-NN con $k = 10$, 50 y 100; son 100 consultas de cada tipo, con centros tomados de la misma muestra. La búsqueda secuencial y el R-Tree se miden ejecutando la consulta SQL completa en el gestor. Para GiST se usó PostGIS con una columna `geography`, `ST_DWithin` para el rango y el operador `<->` para k-NN.

Validación de los resultados. Antes de comparar tiempos se verificó que las tres técnicas devuelven lo mismo: el script compara las filas del R-Tree con las de la búsqueda secuencial y con las de PostGIS, y las 1 800 consultas coincidieron. Además, con `EXPLAIN` se comprobó que PostgreSQL usa el índice GiST en todas.

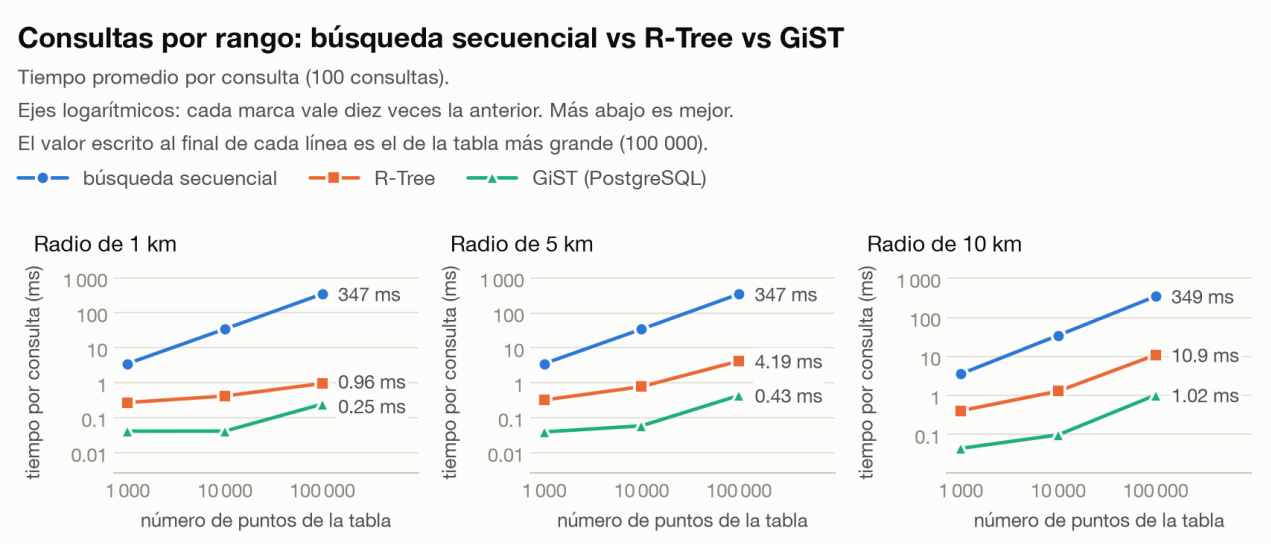


Figura 6: Consultas por rango: tiempo promedio por consulta para radios de 1, 5 y 10 km.

Tabla 8: Consultas por rango. Tiempo promedio por consulta en milisegundos (promedio de 100 consultas). Un valor menor es mejor; el mejor de cada tamaño está en negrita. La última columna es el número de puntos que devuelve en promedio cada consulta con la tabla de 100 000.

		Número de puntos de la tabla			
Consulta	**Técnica**	**1 000**	**10 000**	**100 000**	**Resultados**
Radio de 1 km (ms)	búsqueda secuencial	3.52	34.6	347	64.1
	R-Tree	0.27	0.42	0.96	
	GiST (PostgreSQL)	**0.041**	**0.041**	**0.25**	
Radio de 5 km (ms)	búsqueda secuencial	3.53	34.4	347	544
	R-Tree	0.32	0.78	4.19	
	GiST (PostgreSQL)	**0.039**	**0.059**	**0.43**	
Radio de 10 km (ms)	búsqueda secuencial	3.58	34.8	349	1 577
	R-Tree	0.41	1.29	10.9	
	GiST (PostgreSQL)	**0.044**	**0.098**	**1.02**	

Tabla 9: Consultas k-NN. Tiempo promedio por consulta en milisegundos (promedio de 100 consultas). Un valor menor es mejor; el mejor de cada tamaño está en negrita.

		Número de puntos de la tabla		
Consulta	**Técnica**	**1 000**	**10 000**	**100 000**
k-NN, $k = 10$ (ms)	búsqueda secuencial	6.25	75.1	1 306
	R-Tree	0.34	0.44	0.60
	GiST (PostgreSQL)	**0.055**	**0.056**	**0.12**
k-NN, $k = 50$ (ms)	búsqueda secuencial	6.23	75.8	1 320
	R-Tree	0.55	0.69	0.88
	GiST (PostgreSQL)	**0.088**	**0.094**	**0.18**
k-NN, $k = 100$ (ms)	búsqueda secuencial	6.38	76.4	1 296
	R-Tree	0.80	1.00	1.21
	GiST (PostgreSQL)	**0.12**	**0.13**	**0.24**

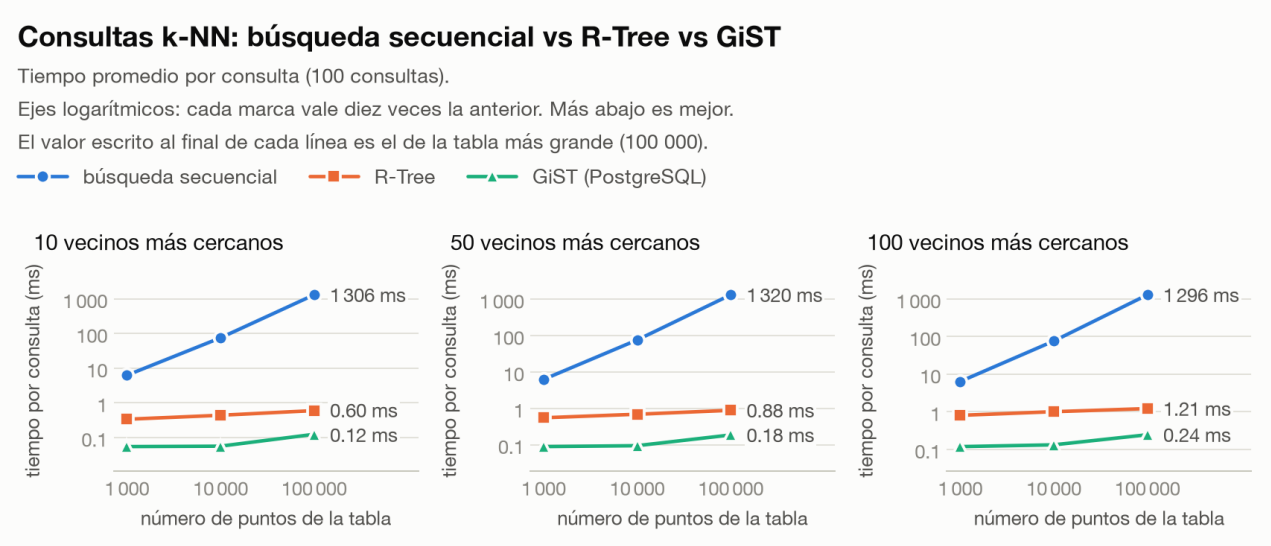


Figura 7: Consultas k-NN: tiempo promedio por consulta para $k = 10$, 50 y 100.

Nodos visitados. El plan de ejecución indica cuántos nodos del R-Tree visitó cada consulta (tabla 10). El árbol pasa de 7 a 606 nodos al crecer la tabla, pero la consulta k-NN con $k = 10$ pasa de visitar 2.3 nodos en promedio a visitar 3.9. Eso explica que su tiempo casi no aumente.

Tabla 10: Nodos del R-Tree visitados en promedio por consulta (promedio de 100 consultas) y total de nodos del árbol.

	Número de puntos de la tabla		
Consulta	**1 000**	**10 000**	**100 000**
Radio de 1 km	2.0	2.2	4.3
Radio de 5 km	2.1	2.9	9.4
Radio de 10 km	2.3	3.8	16.9
k-NN, $k = 10$	2.3	2.4	3.9
k-NN, $k = 50$	2.9	3.2	4.8
k-NN, $k = 100$	3.4	4.2	5.8
Total de nodos del árbol	7	61	606

Construcción, espacio y memoria. La tabla 11 y la figura 8 muestran el costo de construir el índice, el espacio que ocupa y la memoria pico durante las consultas. En la figura, la línea punteada corresponde al mismo R-Tree construido insertando los puntos uno por uno, sin carga masiva.

Tabla 11: Construcción, espacio en disco y memoria del índice espacial. Tiempos en milisegundos (ms), espacio y memoria en KiB. Un valor menor es mejor; el mejor de cada tamaño está en negrita.

		Número de puntos de la tabla		
Medida	**Técnica**	**1 000**	**10 000**	**100 000**
Construcción del índice (ms)	R-Tree, carga masiva	4.17	47.8	712
	R-Tree, inserción uno a uno	142	1 885	22 706
	GiST (PostgreSQL)	**1.79**	**18.4**	**188**
Espacio en disco del índice (KiB)	R-Tree, carga masiva	**32.0**	**248**	**2 428**
	R-Tree, inserción uno a uno	40.0	324	3 260
	GiST (PostgreSQL)	88.0	800	7 792
Memoria pico al consultar (KiB)	búsqueda secuencial	200	772	1 624
	R-Tree	**98.2**	**167**	**1 365**

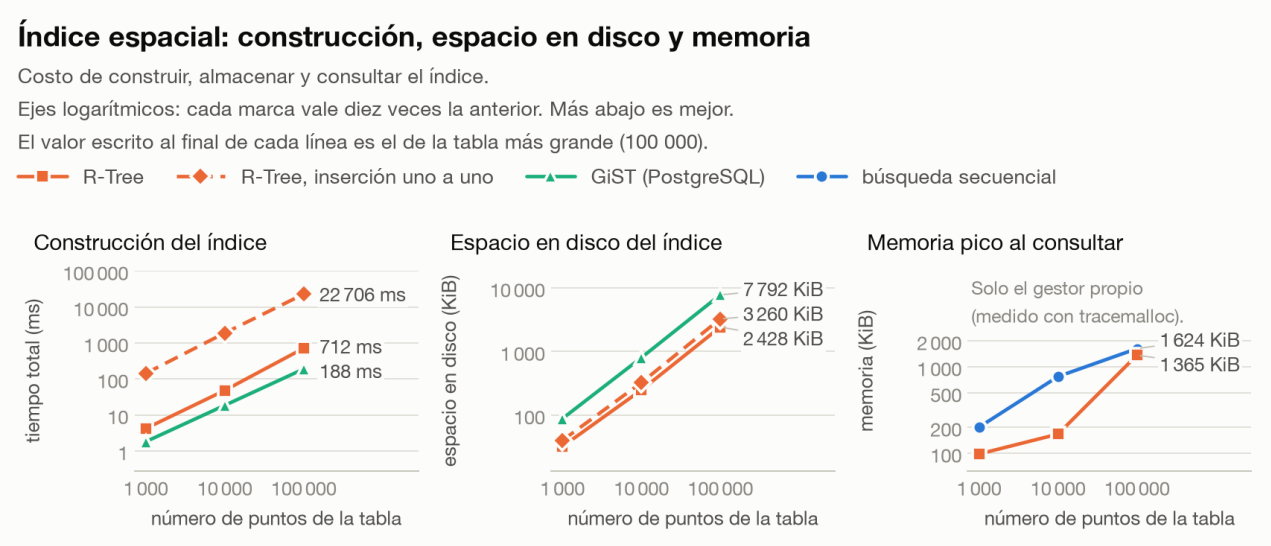


Figura 8: Índice espacial: tiempo de construcción, espacio en disco y memoria pico al consultar.

Análisis.

- El tiempo de la búsqueda secuencial depende del tamaño de la tabla y no del radio: toma 347 ms tanto para 1 km como para 10 km con 100 000 puntos, porque siempre calcula la distancia a todos. El R-Tree toma 0.96 ms para 1 km y 10.9 ms para 10 km; su tiempo depende de cuántos puntos devuelve la consulta.
- Mientras más selectiva es la consulta, mayor es la ventaja del R-Tree: es 364 veces más rápido que la búsqueda secuencial para 1 km, 83 veces para 5 km y 32 veces para 10 km. En este último caso cada consulta ya devuelve el 1.6 % de la tabla.

- En k-NN la diferencia es mayor. Sin índice hay que calcular la distancia a los 100 000 puntos y ordenarlos, lo que toma 1 306 ms para $k = 10$. El R-Tree toma 0.60 ms, 2 192 veces menos. Al pasar de 1 000 a 100 000 puntos el tiempo de la búsqueda secuencial se multiplica por 209 y el del R-Tree solo por 1.8.
- GiST es entre 3.9 y 13 veces más rápido que el R-Tree implementado, pero las dos curvas tienen la misma forma. La diferencia se debe a que GiST está escrito en C dentro de PostgreSQL, mientras que el R-Tree del proyecto está en Python y tiene que deserializar cada nodo que lee.
- La carga masiva (STR) es 32 veces más rápida que insertar los puntos uno por uno y deja un árbol más compacto: 606 nodos en lugar de 814 y un 26 % menos de espacio. El índice del proyecto ocupa 3.2 veces menos que el de PostGIS, que para el tipo `geography` indexa cajas en tres dimensiones.
- En memoria no hay una diferencia importante: con 100 000 puntos el pico es de 1 624 KiB en la búsqueda secuencial y de 1 365 KiB con el R-Tree.

Tabla 12: Resumen del experimento 3: ventajas, desventajas y cuándo usar cada técnica.

Técnica	Ventajas	Desventajas	Cuándo usarla
Búsqueda secuencial	No hay que construir ni mantener un índice.	El tiempo crece con el tamaño de la tabla. En k-NN debe calcular y ordenar todas las distancias.	Tablas pequeñas o consultas poco frecuentes.
R-Tree	Hasta 364 veces más rápido por rango y 2 192 en k-NN con 100 000 puntos. Es el índice que menos espacio ocupa.	Hay que construirlo y mantenerlo. La ventaja baja cuando la consulta devuelve muchos puntos.	Consultas por rango, k-NN o polígono sobre tablas grandes.
GiST (PostgreSQL)	El más rápido: entre 3.9 y 13 veces respecto al R-Tree.	Depende de un servidor externo. Su índice ocupa 3.2 veces más.	Cuando los datos ya están en PostgreSQL. Aquí se usó como referencia.

4.5 Conclusiones

- Ninguna estructura es la mejor en todo. El heap es mejor para insertar y el secuencial para buscar; el B+ agrupado es mejor en consultas y el hash en escrituras; el R-Tree ayuda más mientras más selectiva es la consulta.
- En los índices no agrupados el costo principal es el acceso al heap. Con el mismo árbol B+, el recorrido ordenado es 6.4 veces más lento cuando las hojas guardan direcciones en lugar de las filas.
- El R-Tree evita leer casi todo el árbol: para encontrar los 10 lugares más cercanos entre 100 000 visita en promedio 3.9 de sus 606 nodos.
- Algunos resultados no eran los que se esperaban antes de medir. Por ejemplo, el heap resultó más rápido que el secuencial con 1 000 filas, y el hash más lento que el B+ agrupado en búsquedas por igualdad.

5 Pruebas

El proyecto tiene 1315 pruebas automáticas, que cubren el 99 % de las líneas y ramas del motor. El código pasa además `ruff` y `mypy` en modo estricto. Aparte de las pruebas por caso (estructura vacía, un solo elemento, división y fusión de nodos, desbordamiento de página), se usaron tres formas de verificación:

- *Validadores de estructura.* Cada índice tiene una función que lo recorre completo y comprueba sus invariantes. En el B+ revisa el orden de las claves, la ocupación mínima y que todas las hojas estén al mismo nivel; en el hash, que cada bucket tenga 2^{gd-ld} punteros; en el R-Tree, que cada MBR sea el mínimo.
- *Comparación contra un modelo.* A cada estructura se le aplican miles de operaciones aleatorias y el resultado se compara con una lista ordenada o con una búsqueda por fuerza bruta. Las consultas espaciales se ejecutan con y sin R-Tree y deben devolver lo mismo.
- *Comparación con PostGIS.* Las 1 800 consultas del experimento espacial devuelven las mismas filas que PostGIS.

Las pruebas de almacenamiento usan páginas de 256 bytes para que con pocos registros ya haya varias páginas y se ejerciten las divisiones.